

EduCal

Academic Planning Application

Daniel Santos

Orientadores: Filipe Freitas
Sandra Ferreira

Relatório do projeto realizado no âmbito de Projeto
Licenciatura em Engenharia Informática, Redes e Telecomunicações

Julho de 2026

INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA

EduCal

Academic Planning Application

51701 Daniel Rodrigues Santos

Orientadores: Filipe Bastos de Freitas

Sandra Cristina Pereira Ferreira Neves

Relatório do projeto realizado no âmbito de Projeto
Licenciatura em Engenharia Informática, Redes e Telecomunicações

Julho de 2026

Abstract

ISEL’s academic services currently plan recurring institutional activities, such as application periods, enrolments, exams, and their associated deadlines, using spreadsheets. This process requires the entire calendar to be manually rebuilt every academic year, with every date individually checked against weekends, holidays, and vacation periods, making it slow and prone to error. EduCal is a web application that replaces this manual process with a structured, rule-aware academic planning tool. Events are modeled hierarchically, with each high-level academic objective linked to one or more dated occurrences carrying institutional metadata such as category, status, and responsible department. Its central feature, the Migration Engine, transfers the calendar from one academic year to the next, idempotently cloning events and shifting their dates forward by one year, while a modular Business Rule Engine checks the result against configurable institutional constraints, namely weekends, national holidays, vacation periods, and minimum gaps between events, surfacing violations for manual review rather than silently correcting them. The system is built with Next.js and PostgreSQL, and secures access through a role-based authentication system (Viewer, Editor, Administrator). At the conclusion of the project, event management, the migration engine, the rule engine, and role-based access control are fully implemented and in use. An in-app notification system, reminding users of upcoming occurrences within a configurable lead time, was also delivered, though its originally-scoped companion feature, a manually-sent email template field, and Excel schedule export, were not.

Keywords: academic planning; calendar management; business rule engine; Next.js; PostgreSQL.

Resumo

Os serviços académicos do ISEL planeiam atualmente atividades institucionais recorrentes, como candidaturas, matrículas, exames e os respetivos prazos, recorrendo a folhas de cálculo. Este processo obriga a reconstruir manualmente todo o calendário em cada ano letivo, com cada data a ser verificada individualmente contra fins de semana, feriados e períodos de férias, tornando o processo lento e sujeito a erros. O EduCal é uma aplicação web que substitui este processo manual por uma ferramenta de planeamento académico estruturada e sensível a regras institucionais. Os eventos são modelados de forma hierárquica, em que cada objetivo académico de alto nível está associado a uma ou mais ocorrências datadas, com metadados institucionais como categoria, estado e responsável. A sua funcionalidade central, o Motor de Migração, transfere o calendário de um ano letivo para o seguinte, clonando eventos de forma idempotente e avançando as suas datas em um ano, enquanto um Motor de Regras de Negócio modular valida o resultado face a restrições institucionais configuráveis, nomeadamente fins de semana, feriados nacionais, períodos de férias e intervalos mínimos entre eventos, assinalando violações para revisão manual em vez de as corrigir silenciosamente. O sistema foi desenvolvido com Next.js e PostgreSQL, e protege o acesso através de um sistema de autenticação baseado em papéis (Visualizador, Editor, Administrador). À conclusão do projeto, a gestão de eventos, o motor de migração, o motor de regras e o controlo de acesso baseado em papéis encontram-se totalmente implementados e em uso. Foi também entregue um sistema de notificações na própria aplicação, que alerta os utilizadores para ocorrências próximas dentro de um prazo de aviso configurável; já a funcionalidade complementar prevista no âmbito inicial, um campo de modelo de texto para envio manual por email, e a exportação de calendários em Excel, não foram entregues.

Palavras-chave: planeamento académico; gestão de calendário; motor de regras de negócio; Next.js; PostgreSQL.

Table of Contents

1	Introduction	1
1.1	Motivation	1
1.2	System Requirements	2
1.2.1	Functional Requirements	2
1.2.2	Non-Functional Requirements	2
1.3	Report Organization	3
2	Background and Problem Definition	5
2.1	Problem Statement	5
2.2	State of the Art	6
2.3	Objectives	7
3	System Architecture and Data Model	9
3.1	Global System Overview	9
3.2	Technology Stack	10
3.2.1	Backend	10
3.2.2	Frontend	10
3.3	Data Model and Entity-Relationship Diagram	10
3.3.1	Core Scheduling Entities	12
3.3.2	Classification and Management	12
3.3.3	Automation and Business Rules	12
3.3.4	Notifications	12
3.3.5	Security and Auditing	13
3.4	Data Access Strategy	13
4	Backend	15
4.1	Organization	15
4.2	Authentication	15
4.3	Error Handling	16
4.4	Role-Based Access Control	16
4.5	Layers	17

4.5.1	Controller	17
4.5.2	Service	18
4.5.3	Data	18
4.6	Migration Engine	18
4.7	Business Rule Engine	19
4.7.1	Rule Definitions	20
4.7.2	Built-in Rules	20
4.7.3	Rule Dispatch	21
4.8	Notifications	22
4.9	Tests	22
5	Frontend	23
5.1	Structural Organization	23
5.2	API Communication Architecture	23
5.3	Web Application	24
5.3.1	Navigation and Access Control	24
5.3.2	Notification Bell	24
5.3.3	Authentication	24
5.3.4	Calendar Interface	25
5.3.5	Event Management Dialogs	26
5.3.6	Academic Year Tools	27
5.3.7	User Management Dashboard	28
6	Deployment and Infrastructure	29
6.1	Current Infrastructure	29
6.2	Deployment Roadmap	29
7	Future Work	31
7.1	Schedule Export (RF05)	31
7.2	Email Notification Templates (remaining RF04 scope)	31
7.3	Deployment to the ISEL Virtual Machine	32
8	Conclusion	33
	References	35
A	Detailed Entity-Relationship Diagram	37
B	Selected Code Listings	39
B.1	The Weekend Rule Definition	39
B.2	The Global Rule-Validation Dispatcher	40

B.3	Password Hashing and Verification	41
B.4	The withRolePolicy Authorization Wrapper	42
B.5	The DraggableEvent Component	43
C	Additional Screenshots	45
C.1	Calendar Views	45
C.2	Validation/Migration Results Modal	49

List of Figures

3.1	System architecture diagram.	9
3.2	Simplified Entity-Relationship Diagram, derived from the Drizzle schema. . .	11
4.1	The Controller-Service-Data layering, instantiated for the notification feature.	17
5.1	Login interface.	25
5.2	Main calendar interface (Month view).	26
5.3	Add/Edit Event dialog.	27
5.4	Event Details dialog.	27
5.5	Vacation periods dialog.	28
5.6	User management dashboard.	28
A.1	Full Entity-Relationship Diagram with column-level attributes, derived from the Drizzle schema.	38
C.1	Calendar interface, Agenda view: a scrollable list of occurrences grouped by date or category.	45
C.2	Calendar interface, Day view: a single-day time grid supporting drag-and-drop rescheduling and resizing.	46
C.3	Calendar interface, Week view: the same time-grid interactions as the Day view, spanning a full week.	47
C.4	Calendar interface, Year view: a full-year grid overview of the academic calendar.	48
C.5	Validation/migration results modal: a confirmation banner when no rule violations are found, or one card per violation, each naming the rule, the affected event/occurrence, and the offending date.	49

List of Listings

4.1	Controller layer example.	17
4.2	Date shifting with leap-year clamping.	19
B.1	The Weekend rule definition, in full.	39
B.2	The global rule-validation dispatcher, in full.	40
B.3	Password hashing and verification, in full.	41
B.4	The withRolePolicy authorization wrapper, in full.	42
B.5	The DraggableEvent wrapper component, in full.	43

Chapter 1

Introduction

As part of the Bachelor’s Degree in Computer Science, Networks, and Telecommunications Engineering, this report details the development of the final course project entitled EduCal, an Academic Planning Application. The project, carried out during the 2025-2026 academic year, arises from the need to optimize the management of institutional services that involve critical planning, execution, and improvement activities.

Currently, these activities take place on dates set annually. However, there is no tool to facilitate the transition from one calendar year to the next while respecting specific criteria and deadlines. The current scenario for planning annual tasks is based on dense and complex spreadsheets. Although detailed, these tools are prone to filling errors and require an exhaustive manual effort to replicate the network of objectives, requesters, and instructions in the new school year.

1.1 Motivation

The EduCal project aims to replace this manual process with a web-based application. Unlike general-purpose calendars, which are well suited for personal scheduling but lack specific institutional logic, EduCal is designed specifically for academic planning through several key differentiators.

Primarily, it uses a hierarchical event model where an “Event” represents a high-level academic objective linked to multiple independent “Occurrences,” allowing institutional metadata, such as classification and assigned responsible personnel, to be attached to each entry. Its core functionality is the Migration Engine, which clones the following year’s calendar and shifts each occurrence’s dates forward by one year and runs the configurable business rules against the migrated calendar and reports any violations for manual review. EduCal also includes institutional workflow metadata, such as planning status and responsibility tracking, to support day-to-day management of the calendar.

Given the nature of the project, its initial deployment is expected to take place within ISEL’s services. To this end, the solution was developed using Next.js [1] and PostgreSQL [2],

and designed to be hosted on the institution's internal network. The application implements a decoupled architecture that separates business logic from data storage, allowing the underlying storage technology to change without affecting the rest of the system.

1.2 System Requirements

To ensure the EduCal application effectively addresses the identified problems and achieves its core objectives, a detailed set of requirements was defined. These specifications served as the technical blueprint for the development process, categorizing the system's needs into functional capabilities and non-functional quality standards.

1.2.1 Functional Requirements

Functional requirements outline the specific features, tasks, and behaviors the platform must provide to satisfy user needs. These define exactly what the system does, ranging from the fundamental management of academic events to the automated execution of the migration engine.

- **RF01, Event and Occurrence Management:** the system must allow the creation and management of events that have multiple annual occurrences, each with its own descriptions and dates.
- **RF02, Migration Engine:** it must be possible to transfer the event history from one academic year to the next, automatically validating the resulting dates.
- **RF03, Rule Configuration:** the system must allow flexible business rules (e.g. working days, minimum gaps) to be defined, which the migration engine and event editor must respect.
- **RF04, Notification Templates:** the platform must allow the configuration of email or other text templates associated with events for sending proactive notifications.
- **RF05, Schedule Export:** the system must generate an academic calendar in Excel format for institutional distribution.
- **RF06, Profile Access Control:** multiple user roles must be supported to ensure data security, namely a Viewer with read-only access to consult planning and schedules, and an Editor/Administrator with full permissions to perform CRUD operations, configure rules, and run the migration engine.

1.2.2 Non-Functional Requirements

Non-functional requirements specify the quality attributes, constraints, and performance criteria of the system. Rather than focusing on specific behaviors, they define how the system

performs, ensuring that the application is secure, maintainable, and provides a good user experience.

- **RNF01, Data Integrity:** business days, holidays, and school vacation periods must be validated against the migration process, so that any resulting conflict is reported rather than passed through unnoticed.
- **RNF02, Usability:** the interface must be intuitive and capable of presenting the necessary density of information across multiple calendar views.
- **RNF03, Architecture and Scalability:** the system must be modular and scalable, ensuring fast access to extensive calendars and allowing storage technology to evolve independently of business logic.

1.3 Report Organization

The remainder of this report is organized as follows. Chapter 2 describes the problem, centered on the limitations of ISEL's spreadsheet-based process. Chapter 3 presents the system architecture, technology stack, and data model. Chapter 4 details the backend: its layered organization, authentication and access control, the Migration Engine, the Business Rule Engine, and the notification system. Chapter 5 describes every screen and dialog of the frontend and how it communicates with the backend. Chapter 6 discusses the deployment strategy and current infrastructure, and Chapter 7 covers the parts of the original scope not delivered and the concrete steps toward an institutional deployment. Finally, Chapter 8 summarizes which requirements were delivered and closes the report. Appendices A and B give, respectively, the full column-level Entity-Relationship Diagram and the full source of code shown only abbreviated in the chapters above; Appendix C gives the full-size calendar view and validation/migration screenshots referenced from Chapter 5.

Chapter 2

Background and Problem Definition

This chapter describes the institutional context that motivates the EduCal project, details the concrete problems found in the current process used by ISEL’s academic services, and states the objectives the project sets out to achieve.

2.1 Problem Statement

ISEL’s academic services are responsible for planning and coordinating a large number of recurring institutional activities every academic year, among others application periods, enrolment windows, exam periods, and the various administrative deadlines associated with them. Each of these activities is not a single, isolated event: it typically recurs across the year in multiple occurrences, is owned by a specific department or responsible party, and must respect a set of institutional constraints, such as not falling on a weekend, a national holiday, or a school vacation period.

Today, this planning is done using spreadsheets (Excel). While spreadsheets are flexible and familiar, they are not designed to encode this kind of institutional logic, which creates several concrete problems:

- **No structured relationship between events and their occurrences.** A spreadsheet row cannot natively express that one academic objective (e.g. “Enrolment Period”) has several dated occurrences throughout the year, each with its own metadata; this has to be maintained by hand.
- **No automated year-to-year transition.** Each academic year, the whole calendar has to be manually rebuilt from the previous year’s sheet, with every date recalculated and re-checked against weekends, holidays, and vacation periods.
- **No enforcement of business rules.** Spreadsheets do not validate minimum durations, minimum gaps between events, or conflicts with weekends and holidays; these rules exist only informally and are checked by hand, which makes them easy to miss.

- **Prone to filling errors and duplicated effort.** Replicating the network of objectives, responsible parties, and instructions by hand every year is exhaustive and error-prone: one incorrect date can propagate through the rest of the plan.
- **No institutional workflow metadata.** Classifying events by category, planning status, or responsible department is not natively supported, so this metadata, where it exists at all, is scattered across free-text cells or separate documents.

These limitations directly motivate EduCal, a tool that models academic planning as structured data instead of free-form spreadsheet rows, so that year-to-year transitions, rule checking, and institutional metadata can be handled by software rather than by manual, error-prone repetition.

2.2 State of the Art

Before designing a purpose-built system, it is worth asking whether an existing, general-purpose calendar tool could already solve this problem. The two most used products in this space are Google Calendar and Microsoft Outlook/Calendar, both of which are mature, widely adopted, and already familiar to most staff at an institution such as ISEL.

Both tools are built for scheduling individual meetings and personal appointments, sharing calendars between people, sending invites with RSVP tracking, and booking shared resources such as meeting rooms. Both also support recurring events, generally based on the iCalendar RRULE standard (e.g. “every year on this date”, “every weekday”), which lets a single event repeat automatically according to a fixed pattern.

However, neither tool addresses the specific institutional problem described in Section 2.1:

- **No hierarchical event/occurrence model with institutional metadata.** A recurring event in these tools is just one event that repeats; there is no concept of an academic objective owning multiple independently-manageable occurrences with institution-specific metadata, which is the exact problem EduCal sets out to remove.
- **No academic-year migration.** Standard recurrence rules repeat an event on a fixed pattern; they cannot re-derive a whole calendar of interdependent events for a new academic year while checking each one against that year’s specific holidays and vacation periods. There is no equivalent of EduCal’s Migration Engine.
- **No configurable institutional business rules.** Neither tool can validate, at creation or migration, whether an event falls on a weekend, holiday, or vacation period, or keeps a minimum gap from another related event, the constraints central to EduCal’s Business Rule Engine; users would have to check these manually.

In short, Google Calendar and Outlook are well suited to personal and team scheduling, but they operate at the level of individual events, not at the level of an institutional planning process with hierarchical objectives, yearly migration, and configurable rule checking. This gap is what justifies building EduCal as a dedicated system rather than adopting an existing calendar product.

2.3 Objectives

EduCal aims to replace this manual, spreadsheet-based effort with a web application. The main objectives are:

- **Baseline Creation:** create an academic calendar from scratch, structured around events and their occurrences, as the foundation for the rest of the project's functionality.
- **Automation:** automatically migrate the academic calendar from the current year to the next, instead of rebuilding it by hand every year.
- **Application of Rules:** check compliance with configurable business rules, such as whether events fall on holidays, weekends, or outside school vacation periods, replacing the informal, manual checks used today.
- **Centralization:** unify the planning workflow in a single system that eliminates redundant spreadsheets and ensures data integrity and consistency across the institution.

Chapter 3

System Architecture and Data Model

To ensure the project’s viability, maintainability, and security, the architecture was designed around technologies that are widely adopted in the industry and that allow the application to be hosted within ISEL’s own infrastructure.

3.1 Global System Overview

EduCal follows a three-tier architecture that separates presentation, business logic, and data storage.

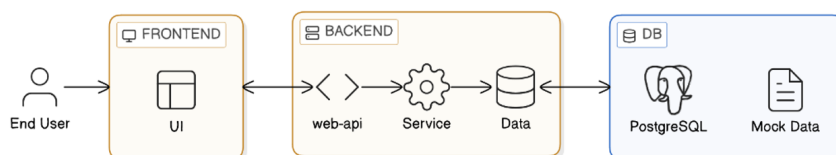


Figure 3.1: System architecture diagram.

As illustrated in Figure 3.1, the system is organized into three layers:

- **Presentation Layer (Frontend):** built with Next.js and React, manages the complex calendar views (Agenda, Day, Week, Month, Year) and specialized interfaces for event/occurrence management, such as dynamic multi-occurrence forms.
- **Logic Layer (Backend):** Next.js API routes backed by a Controller-Service-Data structure (Chapter 4): controllers parse and validate requests, services hold the core business logic and enforce role-based access (the Migration Engine, RF02, and Business Rule Engine, RF03), and the data layer abstracts all storage communication.
- **Data Layer (DB):** a PostgreSQL relational database provides persistent storage for events, occurrences, and system configuration, accessed through the Drizzle ORM.

All components outlined here have been implemented, including PostgreSQL, so far tested only against a local instance (Chapter 6). The data access layer isolates all storage behind a single point in the code, so business services never talk to the database directly, which is what makes it possible to keep an alternative JSON-based backend, currently used for the Vercel preview deployment, without any changes to the business logic, API routes, or frontend (Section 3.4).

3.2 Technology Stack

The stack was chosen to keep the whole application, frontend and backend alike, in a single TypeScript codebase, avoiding the coordination overhead of separate languages or repositories for a project of this size.

3.2.1 Backend

The backend is built on **Next.js** [1] (v16, App Router), using its API routes as the HTTP entry point rather than a separate server framework. Data persistence uses **PostgreSQL** [2] through **Drizzle ORM** [3], a TypeScript-first ORM that provides both a relational query API and raw SQL migrations (**drizzle-kit**). Request payloads are validated end-to-end with **Zod** [4], including schemas that are built dynamically from live database values (e.g. valid event categories). Password hashing uses Node.js’s built-in **crypto** module [5] (**scrypt**), and session cookies follow the **Secure/HttpOnly** guidance from the OWASP Session Management Cheat Sheet [6]. National holiday and business-day calculations use the **date-fns** and **date-holidays** libraries. The automated test suite (Section 4.9) is built on **Vitest** [7].

3.2.2 Frontend

The frontend is built with **React 19** [8] inside the same Next.js application (no separate frontend project), styled with **Tailwind CSS v4** [9] and the **shadcn/ui** [10] component library (built on Radix UI primitives [11]). Forms use **react-hook-form** [12] together with **Zod resolvers** for validation, **sonner** provides toast notifications for asynchronous feedback, **framer-motion** [13] drives the calendar’s animations, the native HTML5 drag-and-drop API handles occurrence rescheduling, and **re-resizable** handles occurrence resizing. The project uses **pnpm** [14] as its package manager and **TypeScript** [15] throughout both the frontend and backend.

3.3 Data Model and Entity-Relationship Diagram

To support automated year-to-year migration, rule validation, and the notification system (Section 4.8), a relational database schema was designed and implemented with Drizzle ORM,

visually outlined in the simplified Entity-Relationship Diagram in Figure 3.2, generated directly from the Drizzle schema definitions (`db/schema/auth.schema.ts`, `db/schema/calendar.schema.ts`, `db/schema/notification.schema.ts`, `db/schema/relations.ts`). This diagram shows every table and its relationships but omits column-level attributes for readability; the full version, with every column and its type, is given in Appendix A.

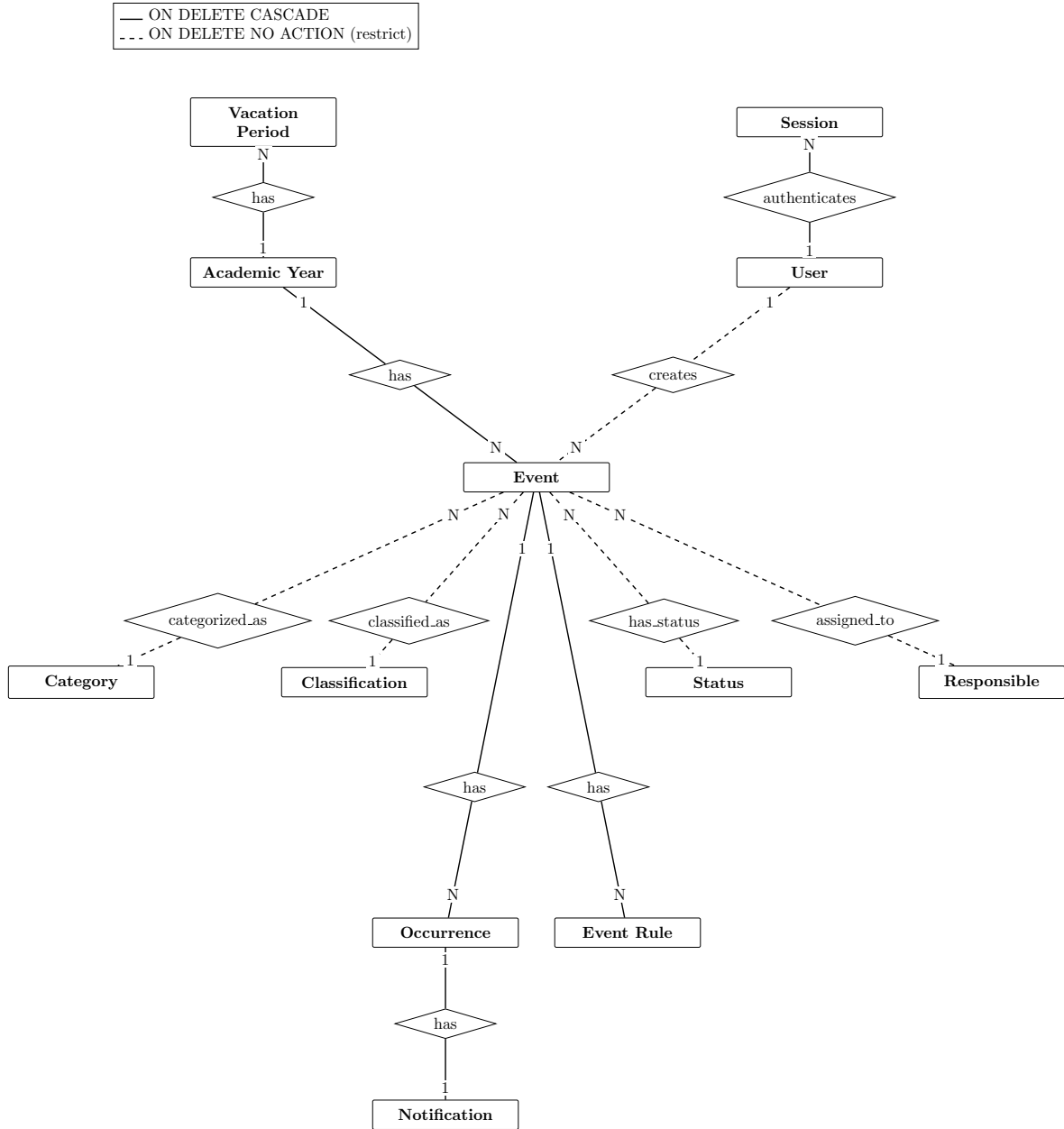


Figure 3.2: Simplified Entity-Relationship Diagram, derived from the Drizzle schema.

Each diamond names the relationship between the two entities it connects, with cardinalities (1/N) marked at each end. Solid lines are foreign keys with **ON DELETE CASCADE**; dashed lines are **ON DELETE NO ACTION** (the referenced row cannot be deleted while still in use). For clarity, this diagram omits the `notification.reads` table, which tracks, per user,

which notifications that user has already read; it is described in Section 3.3.4 and included, with its attributes, in the appendix version of this diagram.

3.3.1 Core Scheduling Entities

The system’s core logic centers on the relationship between events and occurrences, plus the `academic_years` table that anchors both to a specific year:

- **Academic Year:** keyed by its start year, stores a label and the concrete `start_date/end_date` that bound the academic year.
- **Event:** stores high-level metadata such as the objective, category, classification, status, responsible party, and the academic year and user that own it.
- **Occurrence:** a concrete instance of an event, with its own `start_date` and `end_date`, and optional `min_days/min_days_to_next` constraints. Each event can have multiple occurrences, and every occurrence is deleted automatically (`ON DELETE CASCADE`) when its parent event is deleted.

3.3.2 Classification and Management

To facilitate filtering and institutional reporting, the system uses four normalization/lookup tables referenced by events: **Category** (which also stores a `color`, used to render the category badges and filter dots in the calendar UI, Section 5.3.4), **Classification**, **Status**, and **Responsible**. Each event references one value from each of these tables, allowing events to be categorized (e.g. “TFM”, “Candidaturas”), tracked through a planning workflow status (e.g. “Por Fazer”, “Em Curso”, “Feito”), and attributed to an owning department (e.g. “Serviços Acadêmicos”, “Conselho Pedagógico”). Unlike the scheduling entities above, these reference tables are not cascade-deleted: a category, classification, status, or responsible value that is still referenced by an event cannot be removed, which prevents institutional data from silently losing its classification.

3.3.3 Automation and Business Rules

Each event can have zero or more **Event Rules** attached (`event_rules` table), storing a rule type and a JSON `config` object. These rows are what the Business Rule Engine evaluates for that event (Section 4.7); the **Vacation Period** table, scoped to an academic year, stores the school vacation periods that both the rule engine and the migration engine consult.

3.3.4 Notifications

The **Notification** table (`notifications`) stores one row per occurrence a notification has been generated for, uniquely keyed by `occurrence_id` so at most one exists per occurrence at

a time, together with the occurrence's start date and lead time as they were at generation, `occurrence_start_date_at_gen` and `lead_days_at_gen`, which is how the service (Section 4.8) detects that a notification has gone stale and needs regenerating. Rather than targeting a single user, a notification is shared institution-wide; per-user read state is tracked separately in **Notification Read** (`notification_reads`), a table with a composite primary key of `notification_id` and `user_id` plus a `read_at` timestamp, so the same notification can be read by one user and still show as unread to another. Both tables cascade-delete: a Notification is removed when its Occurrence is deleted, and a Notification Read row is removed when either its Notification or its User is deleted.

3.3.5 Security and Auditing

The **User** and **Session** entities manage multi-role access control. The User table stores a `script`-hashed password and a role (`viewer`, `editor`, or `admin`); the Session table tracks active session tokens and their expiration, and is deleted automatically when its owning user is deleted. Every Event additionally stores the `user_id` of its creator, providing a basic audit trail of who owns each entry in the calendar.

3.4 Data Access Strategy

The data access layer isolates all storage behind a repository interface per feature (`IAuthRepository`, `ICalendarRepository`, `INotificationRepository`), so that business services only ever call the interface, never a concrete database client. Two concrete implementations exist for each interface: a PostgreSQL implementation built on Drizzle ORM (`AuthRepositoryPostgres`, `CalendarRepositoryPostgres`, `NotificationRepositoryPostgres`), intended for the institutional VM deployment (Chapter 6), and a JSON-backed implementation on top of Upstash Redis [16] (`AuthRepositoryRedis`, `CalendarRepositoryRedis`, `NotificationRepositoryRedis`), used for the Vercel [17] preview deployment and local development, since it requires no remote database server to be provisioned.

Which implementation is used is decided by a small factory, selected through the `DATALAYER` environment variable: any value other than the literal `"redis"` (including the variable being unset) resolves to PostgreSQL. Because every business service depends only on the repository interface, switching between the two backends requires no change to controllers, services, API routes, or the frontend; only the environment variable changes. This design was validated in practice during development, when the project moved from the JSON-backed mock data used while building the business logic to a locally-tested PostgreSQL database without touching the migration engine, the rule engine, or any UI code.

Chapter 4

Backend

This chapter details the backend implementation of EduCal, covering its layered organization, authentication, authorization, error handling, and testing approach.

4.1 Organization

The backend is organized per feature (`auth`, `calendar`, `notifications`) under `server/`, each following the same three-layer structure of Controller, Service, and Data, plus a `server/shared/` module for cross-cutting concerns (the API response envelope, domain error type, and the data-layer factory). This mirrors the Controller-Service-Repository pattern used since the initial implementation: controllers such as `authController` and `calendarController` act as orchestrators that parse and validate the incoming HTTP request and hand off to the corresponding service, keeping the route handlers in `app/api/**/*.route.ts` as thin, near-identical wrappers regardless of feature or HTTP verb, for example `export const POST = withApiAuth(async (request) => { return calendarController.createEvent(request); })`. Neither the route nor the controller action parses the request itself, checks the role, or catches errors directly: `withApiAuth` (Section 4.2) resolves the session cookie into a user before the handler runs, and every failure, whether it comes from validation, authorization, or the database, is funnelled through the controller’s own `fail()` call (Section 4.3).

4.2 Authentication

Passwords are never stored in plain text: they are hashed with Node.js’s built-in `crypto` module [5] using `scrypt`, a per-password random 16-byte salt, and a 64-byte derived key, encoded as `scrypt$salt$hash` and stored in `users.password_hash`. Verification re-derives the hash from the candidate password and the stored salt, then compares it against the stored hash with a constant-time comparison (`crypto.timingSafeEqual`) to reduce the risk of timing attacks; the two hashes lengths are compared first, since that call throws on mismatched buffer sizes. The full implementation of both functions is given in Appendix B.3 (Listing B.3).

On login, the service generates a random 48-byte, base64url-encoded session token and persists it, with its expiration, in the `sessions` table: a short-lived session (1 day) by default, or 30 days when the user selects “Remember Me” (`staySignedIn`). The token is sent as a cookie that is `HttpOnly`, `Secure` in production, and `SameSite=Lax` [6], carrying a `maxAge` only when “Remember Me” is selected. Every authenticated request also re-validates the session server-side against its stored expiration, deleting it if expired.

4.3 Error Handling

API responses follow a consistent envelope: `{data: ...}` on success, `{error: {code, message, details}}` on failure. Domain-level failures are raised as a typed `DomainError` (an error code such as `UNAUTHORIZED`, `FORBIDDEN`, `NOT_FOUND`, or `CONFLICT`, plus an HTTP status), and Zod validation failures become a `VALIDATION_ERROR` with a structured, field-level breakdown. Every controller method funnels its errors through this single `fail()` helper, so the frontend always receives a predictable error shape regardless of which layer raised it.

4.4 Role-Based Access Control

Three roles are supported, `viewer`, `editor`, and `admin`, ordered in a strict hierarchy. Calendar-editing operations (creating, updating, or deleting events, occurrences, and vacation periods, and running the migration engine) require at least the `editor` role; user management requires the `admin` role. Authentication itself is enforced by HTTP-level middleware (`withApiAuth/withApiAuthContext`) that resolves the session cookie into a user and rejects unauthenticated requests before they reach a controller; role checks are then applied one layer down, inside the services.

Rather than scattering an `if (role !== "editor") throw ...` check across each service method, role enforcement is centralized in a single generic wrapper, `withRolePolicy`, applied once per service. It takes the service instance, a policy map from method name to a minimum role (or `"any"`), and a `getRole` function reading the caller’s role off that method’s own arguments (Section 4.5.1), and returns a new object where every listed method asserts the minimum role before delegating to the original. The design is deny-by-default at the type level, not only at runtime: a method the policy map omits is absent from the returned object’s type entirely, so adding a service method without a matching policy entry fails to compile at every call site. `calendarService` and `authService` are both exported this way, e.g. `listEvents: "any"` alongside `createEvent: "editor"`; the full implementation is in Appendix B.4 (Listing B.4).

Beyond the basic role hierarchy, the system enforces three self-management invariants to prevent an institution from locking itself out of administration:

- an administrator cannot demote their own account away from `admin`;

- an administrator cannot delete their own account;
- the last remaining administrator cannot be demoted or deleted; the service counts existing admins before allowing either operation and rejects it if only one would remain.

4.5 Layers

Every backend feature is organized into the same three layers, Controller, Service, and Data, each with a single, narrow responsibility and depending only on the layer directly below it. Figure 4.1 illustrates this flow for a single request, and the rest of this section walks through each layer in turn, using the notification feature’s own controller, service, and repository as a concrete, small enough example to show in full.

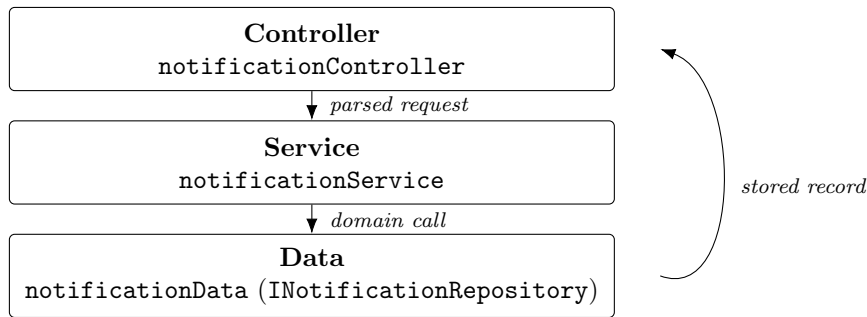


Figure 4.1: The Controller-Service-Data layering, instantiated for the notification feature.

4.5.1 Controller

Controllers (`server/auth/controllers/`, `server/calendar/controllers/`, `server/notifications/controllers/`) are the entry point for each API route. They parse and validate route parameters and the request body, invoke the corresponding service method, and translate the result, or any error thrown, into an HTTP response using the shared `ok()`/`fail()` helpers. This is also where the session cookie is set on login and cleared on logout. Listing 4.1 shows the notification controller’s `markRead` action, a minimal case of this shape.

```

1 async markRead(request: IRequestWithAuth, notificationIdValue: string) {
2   try {
3     const notificationId = parseNotificationId(notificationIdValue);
4     await notificationService.markRead(request, notificationId);
5     return ok({ read: true });
6   } catch (error) {
7     return fail(error);
8   }
9 },

```

Listing 4.1: Controller layer example (`server/notifications/controllers/notification.controller.ts`).

The method does no business logic of its own: it validates that a notification id was actually supplied, delegates the read entirely to `notificationService.markRead`, and converts

either outcome into the standard response envelope (Section 4.3). The `try/catch` boilerplate is identical across every controller action in the codebase, which is precisely what keeps the layer thin: nothing here depends on how a notification is read or where it is stored.

4.5.2 Service

Services (`server/auth/services/auth.service.ts`, `server/calendar/services/calendar.service.ts`, `server/notifications/services/notification.service.ts`) hold all business logic and authorization checks, and are the bridge between the controllers and the data layer. This is where the RBAC invariants described in Section 4.4 are enforced, and where the central pieces of calendar business logic live: the Migration Engine (Section 4.6), the Business Rule Engine (Section 4.7), and the notification lead-time computation (Section 4.8), which decides whether a notification is due for a given occurrence by comparing its effective lead time, in working days, against the current date each time notifications are read, rather than on a schedule.

4.5.3 Data

The data layer (`server/auth/data/`, `server/calendar/data/`, `server/notifications/data/`) exposes a repository interface per feature (`IAuthRepository`, `ICalendarRepository`, `INotificationRepository`), abstracting all storage access; `INotificationRepository`, for instance, is six methods, `listAll`, `insertMany`, `deleteManyByOccurrenceIds`, `listReadNotificationIdsForUser`, `markRead`, and `markAllRead`, with no mention of SQL or Redis anywhere in it. Services depend only on this interface, never on a concrete database client, which is what enables the pluggable PostgreSQL/Redis backend described in Section 3.4: `NotificationRepositoryPostgres` implements it directly against the `notifications/notification_reads` tables (Section 3.3) with Drizzle queries, and `NotificationRepositoryRedis` implements the same interface against a single JSON blob in Upstash Redis.

4.6 Migration Engine

The migration of the academic calendar from one year to the next is implemented as a single service operation (`CalendarService.migrateAcademicYear`), exposed to editors and administrators through dedicated “Validate” and “Migrate” actions in the calendar header (Section 5.3.6).

- **Vacation periods first:** for the target academic year (source year + 1), the engine migrates vacation periods before events, so subsequent rule checks against the target year already see the correct vacation windows.
- **Idempotent event migration:** every source-year event is fingerprinted from a fixed subset of its fields (excluding database-generated ids); an existing equivalent in the

target year is skipped, otherwise the event is cloned, so the operation is safe to run more than once.

- **Date shifting and leap years:** each occurrence’s dates shift forward one calendar year; the 29th of February is clamped back to the 28th when the target year is not a leap year.
- **Validation, not automatic correction:** immediately after migrating, the full business rule set (Section 4.7) runs against the result, and any violations are collected into a report rather than auto-corrected, left as a manual step guided by the issue report shown in the application (Section 5.3.6). The same logic powers the standalone, read-only “Validate” action.

Listing 4.2 shows `shiftDateByYears`, the pure function every occurrence’s start and end dates are passed through during migration, which implements the leap-year clamping described above.

```
1 export const shiftDateByYears = (dateValue: string, years: number): string => {
2   const source = new Date(dateValue);
3   const target = new Date(dateValue);
4   target.setUTCFullYear(source.getUTCFullYear() + years);
5
6   if (target.getUTCMonth() !== source.getUTCMonth()) {
7     target.setUTCDate(0);
8   }
9
10  return target.toISOString();
11 };
```

Listing 4.2: Date shifting with leap-year clamping (`shared/calendar/academic-year.ts`).

Adding one year to a date is not simply incrementing its year component: calling `setUTCFullYear` on the 29th of February of a leap year silently rolls the date forward into the 1st of March whenever the target year is not itself a leap year, since the 29th of February does not exist there. The function detects this by checking whether the resulting month still matches the source month; if it does not, the date is rolled back with `setUTCDate(0)`, which JavaScript resolves to the last day of the previous month, that is, the 28th of February. Every other date is left unaffected, since shifting a day that exists in every year by whole years never changes its month.

4.7 Business Rule Engine

A modular rule engine validates event occurrences against configurable institutional constraints. Each rule is a self-contained unit implementing a shared interface; the engine itself has no knowledge of what a rule checks, it only calls the rule’s `validate` function and collects the result. Rules are registered in a single, central list the engine iterates over, so a new institutional rule is added by writing one file and registering it, with no branching logic

and no changes to the migration or validation code. This same registry also drives the rule-configuration UI in the event form (Section 5.3.5), which reads each rule’s field metadata to render the matching controls.

4.7.1 Rule Definitions

Every rule definition has the same four parts:

- **id**: the stable key stored in `event_rules.type` and used to dispatch validation.
- **fields**: the rule’s configurable inputs and their defaults, read by the event form to render matching controls with no rule-specific UI code.
- **translations**: every user-facing string the rule needs, one display label plus one message per violation kind, keyed by locale.
- **validate**: a pure function taking the event, one occurrence, and the rule’s configuration, returning zero or more violations, each carrying a message key and the field/date that triggered it, rather than a pre-formatted string, so the frontend renders every rule consistently (Section 5.3.6).

The Weekend rule (Appendix B.1, Listing B.1) is a representative example: it flags an occurrence whose start date, end date, or any spanned date falls on a Saturday or Sunday. Weekend, Holiday, and Vacation period share the same three independent toggles, `checkStart`/`checkEnd`/`checkRange`, differing only in *what* counts as a match on a given day; this shared loop is factored into one generic helper, `runDayRangeCheck` (`server/calendar/rules/utils/day-range-check.ts`), that each rule supplies with:

- **matchDay**: a single-day predicate (“is this a weekend”, “a holiday”, “inside a vacation period”), tested against the start date, the end date, and, for `checkRange`, every day in between, stopping at the first match.
- **toViolation**: builds the rule’s own `messageParams` from whatever `matchDay` matched.

An optional `matchRange` lets a rule replace that day-by-day walk with a single whole-interval check instead, which Vacation period uses since a vacation period is a continuous range rather than a set of independently meaningful days.

4.7.2 Built-in Rules

Seven rules are currently implemented:

- **Weekend**: flags occurrences whose start, end, or intermediate dates fall on a Saturday or Sunday.

- **Holiday:** flags occurrences that coincide with Portuguese national holidays.
- **Vacation period:** flags occurrences that overlap a registered school vacation period for that academic year.
- **Event no-overlap:** flags occurrences whose date span overlaps other, user-selected events.
- **Minimum gap to event:** enforces a minimum working-day gap between one event’s end and a related, user-selected event’s start.
- **Minimum duration:** always-on, enforces a minimum calendar-day duration, driven by the occurrence’s own `minDays`.
- **Minimum gap to next occurrence:** always-on, enforces a minimum working-day gap between consecutive occurrences, driven by `minDaysToNext`.

The first five rules are selectively enabled per event through the event editor; the last two are always active and not exposed in the rule-configuration UI (marked `hidden` in the rule metadata). All rules are advisory: violations are surfaced for manual review, not enforced automatically, distinct from the structural, blocking validation Zod performs on every API request.

Most rules derive everything they need from the event and occurrence being validated, plus `context.allEvents`, for cross-event checks such as Event no-overlap. Vacation period is the exception: it consults `context.vacations` instead, a list populated from the per-academic-year Vacation Period table (Section 3.3.3), managed through its own dialog (Section 5.3.6). Its `matchRange` tests whether the occurrence’s whole span overlaps a vacation period, and every violation carries a `messageParams.vacationLabel` naming the period matched, e.g. “The end coincides with the vacation period ‘Christmas break’”. Holiday follows the same shape, using `date-holidays`’ `isHoliday` lookup as `matchDay`.

4.7.3 Rule Dispatch

Individual rule definitions are only half of the engine: something still has to decide, for every occurrence of every event, which rules apply and collect their results into a single report. That is `CalendarService.collectEventRuleIssues` (Appendix B.2, Listing B.2), the actual entry point of the engine that both `validateAcademicYear` and `migrateAcademicYear` (Section 4.6) call to produce their issue reports.

A single `context` object (`allEvents/academicYearStart/vacations`) is built once per event and shared across every rule invocation for it, so cross-event rules never fetch their own data. Occurrences are sorted chronologically first, and a small shared closure flattens every violation into a self-contained issue, so both the always-on built-in rules and every explicitly

attached rule share the same construction logic instead of repeating it. Because `validateAcademicYear` and `migrateAcademicYear` both call this exact method, the read-only “Validate” check and the automatic check after a migration can never disagree about what counts as a violation.

4.8 Notifications

Beyond validating dates against institutional rules, EduCal reminds users of occurrences coming up soon, through an in-app notification system rather than the email-based templates originally scoped under RF04 (Section 1.2.1); the remaining, undelivered part of that requirement is discussed in Chapter 7. Each event carries a default lead time, `notifyDaysBefore`, and any occurrence may override it with `notifyDaysBeforeOverride`, both set in the event/occurrence form (Section 5.3.5). `ensureGenerated` runs lazily, on every read, comparing the current date against each occurrence’s effective lead time in working days (Section 4.5.2), rather than on a schedule. A notification, once generated, is a single row shared by every user, keyed uniquely by `occurrenceId`; read status is tracked separately, per user, in `notification_reads` (Section 3.3), so marking one read for one user never affects whether another still sees it as unread. If an occurrence’s date or lead time changes after generation, the stale record is deleted and regenerated, and notifications for deleted occurrences are cleaned up the same way.

4.9 Tests

An automated test suite, built with Vitest [7], covers the system’s pure-logic layer: the business rules and the rule dispatcher (Section 4.7), the working-day calculation utility, password hashing/verification, the `withRolePolicy` wrapper (Sections 4.2, 4.4), the academic-year date helpers (Section 4.6), and the notification service’s lead-time logic (Section 4.8, exercised against an in-memory fake of its repository). It comprises 13 test files and 79 tests, all passing; `vitest.config.ts` scopes coverage measurement to exactly the modules listed above (rules and rule dispatcher, `crypto.ts`, `academic-year.ts`, `authorize.ts`, and `notification.service.ts`), reaching approximately 97% statement coverage across all five (`@vitest/coverage-v8`). The test process’s timezone is pinned to UTC and cases use fixed calendar dates, including real Portuguese holidays, so results never depend on the host’s timezone or the current date.

The service layer, the data-access layer (PostgreSQL and Redis), API routes, and the frontend are covered by manual testing instead: exercising each requirement’s flows through the UI and, for API endpoints, direct HTTP requests. The application was also tried by a small group of ISEL academic staff on their own initiative, with generally positive feedback leading to several UI adjustments.

Chapter 5

Frontend

This chapter describes the frontend implementation of EduCal: how the codebase is organized, how it communicates with the backend, and every screen and dialog that makes up the application.

5.1 Structural Organization

EduCal is a single Next.js application (App Router) that hosts both frontend and backend, rather than two separate projects. The `app/` directory is kept as a thin routing layer (the calendar home page, the login page, the admin users page, and the API route handlers under `app/api/`), while the actual UI is organized by feature under `features/`: `features/calendar/` (the calendar itself, including views, dialogs, header controls, drag-and-drop, and settings), `features/auth/` (the login form), `features/admin-users/` (the user management dashboard), and `features/notifications/` (the notification bell, Section 5.3.2). Reusable, feature-agnostic UI primitives (buttons, dialogs, forms, selects, etc.) live under `components/ui/`, sourced and adapted from `shadcn/ui` rather than installed as a compiled dependency, and code shared between client and server (types, Zod schemas, role logic) lives under `shared/`.

5.2 API Communication Architecture

EduCal deliberately does not use HTTP API routes for reading data. Server components fetch data directly by calling backend services: `features/calendar/requests.ts`, marked "`server-only`", is invoked once from the `Calendar` server component, and the result (events, users, enum values, vacations) is passed down as initial props into a React context provider (`CalendarProvider`), avoiding a network round-trip for the common case of simply loading the calendar.

Mutations (creating or editing an event, logging in, managing users) do go through HTTP: a shared fetch wrapper (`lib/api-client.ts`) attaches the JSON content type, unwraps the `{ data: ... }` envelope on success, and on failure parses the backend's error envelope

(Chapter 4) into a user-facing, translated error message. Feature-specific request modules build on this wrapper per mutation endpoint. Components call these functions inside a `try/catch` and surface the result as a **sonner** toast, giving immediate feedback without a full page reload.

5.3 Web Application

This section walks through the application as a signed-in user experiences it: how routes are protected, how a session is established, and every screen, dialog, and persistent UI element (such as the notification bell, Section 5.3.2) that makes up the calendar itself.

5.3.1 Navigation and Access Control

The application has three main routes: the calendar (`/`), the login page (`/login`), and the admin users page (`/admin/users`). Route-level access control is handled entirely on the server, in the page component itself, rather than by a client-side auth context: `app/page.tsx` resolves the current session and redirects to `/login` if none exists, before any calendar markup is ever sent to the client; `app/login/page.tsx` redirects an already-authenticated user straight to `/`; and `app/admin/users/page.tsx` redirects to `/login` if unauthenticated and to `/` if the current user is not an `admin`. On the calendar page, once the session check passes, the actual event/user/enum/vacation data is fetched inside the `Calendar` component behind a `Suspense` boundary, which shows a loading skeleton while it resolves.

5.3.2 Notification Bell

Once authenticated, every page shows a bell icon in the global navbar, backed by the notification system described in Section 4.8. `NotificationBell` polls `GET /api/notifications` every 60 seconds and keeps an unread-count badge on the icon; opening it lists each unread notification’s occurrence description and date, with a “mark all read” shortcut. Clicking one marks it read and navigates to `/?openEventId=...&openOccurrenceId=...`; `NotificationEventOpener` watches those parameters and, once resolved against the loaded calendar context, opens that event directly in the read-only Event Details dialog (Section 5.3.5), skipping any manual search.

5.3.3 Authentication

Figure 5.1 shows the login form, which has three fields: `email`, `password`, and a “Remember Me” checkbox (`staySignedIn`, checked by default) that controls how long the session cookie persists (Section 4.2). Fields are validated client-side with the same Zod schema shape used server-side, highlighting invalid fields in red. On submit, the button switches to a disabled “submitting...” state; on success the app navigates back to the calendar and refreshes its

server components so the newly authenticated session is picked up immediately; on failure, the server’s error message is shown inline below the form, rather than as a toast.

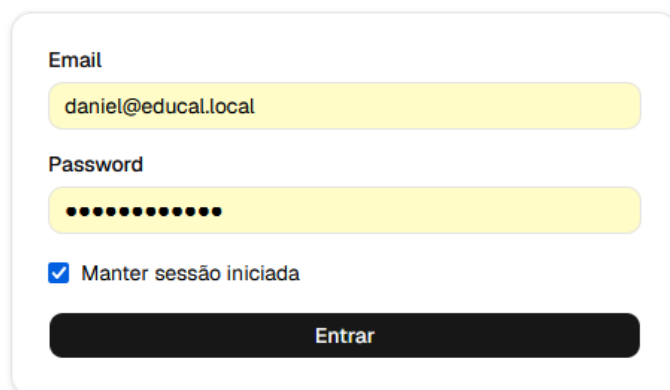
A login form with a light gray border and rounded corners. It contains three main sections: an 'Email' section with a yellow input field containing 'daniel@educal.local'; a 'Password' section with a yellow input field filled with black dots; and a checkbox labeled 'Manter sessão iniciada' which is checked. Below these is a dark gray button with the text 'Entrar' in white.

Figure 5.1: Login interface.

5.3.4 Calendar Interface

The interactive calendar (Figure 5.2) is the core of the application. Its header is split into two rows: a top row with a “Today” shortcut and a date navigator (showing the current month/year, an event-count badge for the visible period, and previous/next controls), and an actions row containing the category filter, the Agenda/Day/Week/Month/Year view switcher, the academic-year tools (Section 5.3.6), a user filter, the “Add Event” button, and the settings menu.

The five views (**Agenda**, **Day**, **Week**, **Month**, **Year**; the latter shown in Figure 5.2, the rest in Appendix C.1) offer different levels of temporal density, from a scrollable list grouped by date or category (Agenda) to a full-year grid overview (Year). Within the day/week grid views, occurrences can be rescheduled directly by dragging them to a different time slot, implemented with the native HTML5 drag-and-drop API, or resized to change their duration, implemented with the `re-resizable` library; both are animated with `framer-motion` and persist immediately through the mutation flow described in Section 5.2. When a month-view day has more occurrences than can be shown inline, an “+N more” control opens a simple list dialog for that day, from which any occurrence can be opened in full detail.

The `DraggableEvent` component wraps every occurrence block shown in the day and week views, connecting the browser’s native drag events to the shared drag-and-drop context (full source in Appendix B.5, Listing B.5). It falls back to rendering its children unwrapped, with no drag handlers, for users without editing rights (`canEditEvents`); otherwise a native `draggable motion.div` dims itself while being dragged and notifies a shared drag context via `onDragStart/onDragEnd`. The drop target and resulting date/time update are handled by the companion `DroppableArea` component and the `updateOccurrence` mutation (Section 5.2).

Two controls narrow down what is shown: a category filter (multi-select dropdown, one entry per `Category`, each with its colored dot) and a user filter (single-select, showing every user’s avatar and name, defaulting to “All users”). A settings menu offers independent toggles for dark/light theme, category badge style (colored block vs. small dot), 12/24-hour time format, and how the Agenda view groups events (by date or by category), all persisted locally per browser.

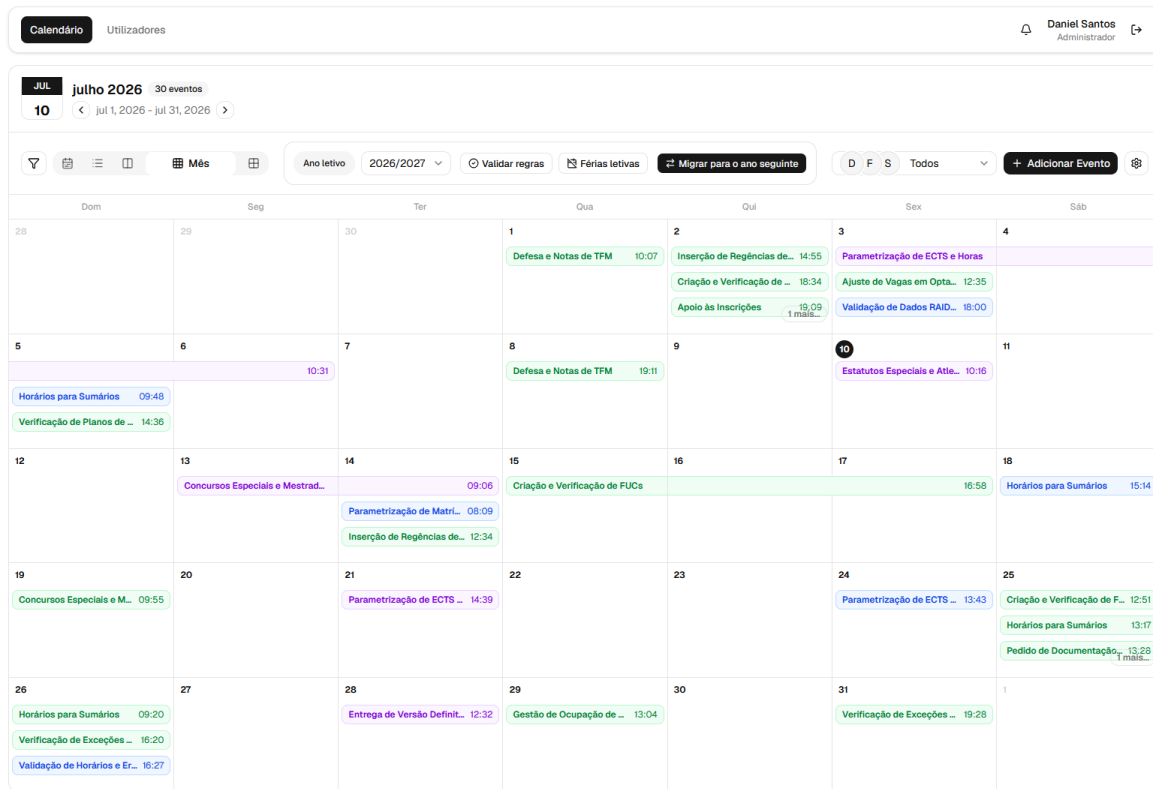


Figure 5.2: Main calendar interface (Month view).

5.3.5 Event Management Dialogs

The **Add/Edit Event dialog** (Figure 5.3) is a single form reused for both creating and editing an event, visible only to editors and above. It has an event-level panel (`name`, `objective`, `category/classification/status/responsible` dropdowns, and a default notification lead time, `notifyDaysBefore`, Section 4.8); an occurrence panel, a repeatable list (at least one required) each with its own `description`, `start/end date-time`, optional min-duration/min-gap constraints, and its own lead-time override (`notifyDaysBeforeOverride`); and a “Rules” panel attaching any of the five configurable business rules (Section 4.7), each rendering its own input controls from that rule’s field metadata.

Figure 5.3: Add/Edit Event dialog.

The **Event Details dialog** (Figure 5.4) is the read-only counterpart, opened when a user clicks an occurrence anywhere in the calendar. It shows the event’s name, category/classification/status badges, its objective, who created it, the selected occurrence’s dates, every occurrence in a scrollable grid, and a summary of the rules attached to the event, in the same terms an end user configured them in. Users with edit rights see shortcuts here to open the Edit dialog or delete the event directly.

Figure 5.4: Event Details dialog.

Deleting an event goes through a dedicated **confirmation dialog** (a destructive-styled button that opens an alert requiring explicit confirmation) before the event and all of its occurrences are removed, since occurrence deletion cascades from the event at the database level (Section 3.3).

5.3.6 Academic Year Tools

The academic-year tools, part of the calendar’s actions bar, let the user pick an academic year and run the migration workflow described in Section 4.6: “Validate” checks the selected year’s events against every business rule without changing anything, and “Migrate” (editors and administrators only) runs the full migration. Both share a results modal (Appendix C, Figure C.5): a confirmation banner if no violations were found, or one card per violation

otherwise, naming the rule, the affected event/occurrence, and the offending date, opening that event’s Details dialog on click. A separate “Vacations” action opens a dedicated dialog (Figure 5.5) for managing that year’s vacation periods, with full create/edit/delete support for editors.



Figure 5.5: Vacation periods dialog.

5.3.7 User Management Dashboard

The administrative dashboard (Figure 5.6), accessible only to administrators, lists every user in a table (name, email, role, profile picture, and actions) and supports full CRUD, with the password field left blank on edit meaning “keep the current password”. Every mutation gives immediate toast feedback, deletions are confirmed before proceeding, and the delete action is disabled for the currently logged-in administrator’s own row, reinforcing the “cannot delete self” invariant already enforced on the backend (Section 4.4).

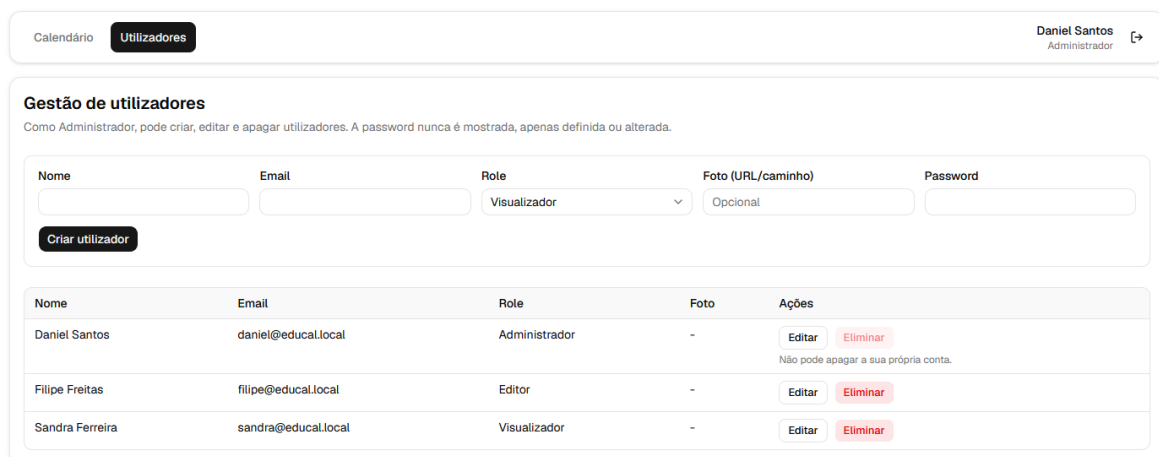


Figure 5.6: User management dashboard.

Chapter 6

Deployment and Infrastructure

6.1 Current Infrastructure

During development, EduCal was run and previewed using **Vercel**, the hosting platform built by the creators of Next.js. It requires minimal configuration for Next.js applications, which made it possible to get a working, publicly reachable preview of each iteration of the application without needing to provision any institutional infrastructure up front.

Provisioning a remote PostgreSQL server for this preview deployment was not warranted at this stage of the project, so the Vercel deployment instead runs on the JSON-blob-backed Redis implementation (`DATA_LAYER=redis`, Section 3.4), avoiding the need for a remote database server entirely. PostgreSQL itself was used and tested only against a local instance throughout development; the ISEL virtual machine described below is intended to be the first environment where it runs continuously.

Locally, the project is run with the standard Next.js toolchain, managed through **pnpm**: `pnpm dev` for the development server, and `pnpm build/pnpm start` to build and run a production Node.js server, with database migrations (`pnpm run db:migrate`) run manually against whichever backend is configured.

6.2 Deployment Roadmap

Because EduCal manages institutional academic planning data, the intended production deployment is a single virtual machine hosted within **ISEL's own internal network**, rather than a third-party cloud platform, so that both the application and its data remain under the institution's direct control. This VM would host both the Next.js application (built and run with `pnpm build/pnpm start`) and the PostgreSQL instance.

A single VM, rather than separate machines for the application and the database, is the intended setup: at the scale EduCal is designed for, a single institution's academic staff, splitting the two would add provisioning and network overhead without the corresponding benefit, independent scaling, that split is meant to buy. Colocating both keeps the surface ISEL's IT staff need to secure down to one machine, at the acceptable cost of the application

and database sharing CPU, memory, and disk.

This step has not been carried out yet: no VM has been provisioned or configured as part of this project's delivery. Setting it up is expected to follow these steps:

1. Receive the VM from ISEL's IT department, provisioned with a Linux distribution, and confirm its network reachability (whether it is reachable only within ISEL's internal network, or also externally, which determines whether a reverse proxy/TLS setup is needed).
2. Install Node.js (the version this project targets), `pnpm`, and a PostgreSQL server on the VM.
3. Create the application's PostgreSQL database and role, and configure `DATABASE_URL` and the remaining environment variables in a `.env` file local to the VM, never committed to version control.
4. Run the Drizzle migrations (`pnpm run db:migrate`) against this new database to create the schema, followed by `pnpm run db:seed` to populate the reference lookup tables (categories, classifications, statuses, responsables) and create the initial administrator account.
5. Build the application (`pnpm build`) and run it as a persistent background service, for example via a process manager such as `pm2` or a `systemd` unit, so that it restarts automatically on crash or VM reboot.
6. Configure a reverse proxy (e.g. Nginx) in front of the Node.js process to handle TLS termination and serve the application on the standard HTTPS port, under whatever internal hostname ISEL assigns it.
7. Set up a recurring backup schedule for the PostgreSQL database (e.g. a nightly `pg-dump` to institutional storage), since this is the point where data loss becomes a real institutional risk once the calendar is in active use.

These steps, along with defining a repeatable deployment process for future updates, are the concrete next steps once the VM is made available by the institution.

Chapter 7

Future Work

This chapter collects the parts of the original project scope that were not delivered within the project timeline, together with the concrete next step for moving the application from its current local/Vercel setup (Chapter 6) into institutional use. Each of these is scoped as a follow-up to the work already completed, rather than a redesign of it: the underlying architecture (Chapter 3) was built to accommodate all three without further structural change.

7.1 Schedule Export (RF05)

RF05 (Section 1.2.1) requires the system to generate an academic calendar in Excel format for institutional distribution; no such export exists in the current codebase. A spreadsheet suits institutional distribution better than a static rendering of the calendar UI would: recipients can sort, filter, and re-organize a schedule of events and occurrences in a way a fixed page layout does not allow. The natural shape for this is one worksheet per academic year, with one row per occurrence and a column for each piece of metadata already modeled on the event and occurrence (name, category, classification, status, responsible, description, and dates), built with a library such as `exceljs`, which produces a native `.xlsx` workbook directly from TypeScript objects. This would follow the same Controller-Service-Data shape as the rest of the backend (Section 4.5): a new service method assembling the relevant events and occurrences for a given academic year and category selection, and a controller action streaming the resulting workbook back as the HTTP response.

7.2 Email Notification Templates (remaining RF04 scope)

RF04 originally called for configurable email or other text templates associated with events, for sending proactive notifications. The in-app half of this requirement was delivered (Section 4.8): EduCal now surfaces upcoming occurrences to signed-in users through the notification bell (Section 5.3.2), based on a configurable per-event or per-occurrence lead time. What remains is the template half of the original requirement: a free-text field, attached to

an event, that an editor could fill in with the wording of a reminder (for example, the exact phrasing to send to a department ahead of an enrolment deadline), and a way to surface that text so a staff member could copy it and send it manually through the institution’s own email system; it remains future work purely because it was not reached within the project timeline, not because of any technical obstacle identified during development.

7.3 Deployment to the ISEL Virtual Machine

The application currently runs from a local development environment and a Vercel preview deployment backed by the Redis-based data layer (Section 6.1); PostgreSQL, the storage backend the institutional deployment is meant to use, has only been exercised locally. Chapter 6 already lays out the intended production setup, a single virtual machine on ISEL’s internal network hosting both the Next.js application and PostgreSQL, and the concrete steps to provision it (Section 6.2). None of those steps have been carried out yet, because of timeline constraints but, the immediate next steps are running the Drizzle migrations against the new database, seeding its reference lookup tables and initial administrator account, and putting the application behind a process manager and reverse proxy so it survives restarts and serves traffic over HTTPS. Establishing a recurring PostgreSQL backup schedule at that point, rather than after the calendar is already in active use, would close the last gap between the current delivery and a production-ready deployment.

Chapter 8

Conclusion

At the conclusion of the project timeline, EduCal has moved from a conceptual set of requirements to a functional web application that replaces the spreadsheet-based process previously used for institutional academic planning at ISEL (Chapter 2). Of the six functional requirements defined in Section 1.2.1, four were fully implemented and are in use: hierarchical event and occurrence management (RF01, Sections 3.3.1 and 5.3.5), the idempotent, rule-aware academic-year Migration Engine (RF02, Section 4.6), the configurable Business Rule Engine (RF03, Section 4.7), and role-based multi-user access control (RF06, Sections 4.2, 4.4, and 5.3.7). Notification support (RF04) was partially delivered: the in-app notification bell (Sections 4.8 and 5.3.2), reminding users of upcoming occurrences within a configurable lead time, is implemented and in use, but the requirement's other half, a manually-sent email template field, was not, and neither was Excel schedule export (RF05); both are discussed as concrete next steps in Chapter 7.

The decision to structure the backend as a decoupled Controller-Service-Data pipeline, with all storage access hidden behind a repository interface (Section 3.4), proved effective throughout the project. It allows the migration engine, the rule engine, and the notification system alike to be developed and iterated on independently of the underlying data store, and it makes the transition from the JSON-backed mock data used while developing the business logic to the production PostgreSQL database a matter of changing a single environment variable, with no changes required to the controllers, services, API routes, or frontend. The same separation is what makes it possible to keep the lightweight Redis-backed implementation alongside PostgreSQL for development and demos, without either implementation leaking into the rest of the system.

Overall, EduCal addresses the core problem stated in Chapter 2, the manual, error-prone, spreadsheet-based rebuilding of the academic calendar every year, through automated migration and configurable rule checking, backed by a secure, role-based multi-user system and proactive in-app reminders. The remaining gaps, the email template field, schedule export, and the ISEL VM deployment, are detailed in Chapter 7.

References

- [1] Vercel. Next.js documentation, 2026. [Online; accessed April 2026].
- [2] The PostgreSQL Global Development Group. Postgresql documentation, 2026. [Online; accessed June 2026].
- [3] Drizzle Team. Drizzle orm documentation, 2026. [Online; accessed July 2026].
- [4] Colin McDonnell. Zod documentation, 2026. [Online; accessed April 2026].
- [5] OpenJS Foundation. Node.js crypto module documentation, 2026. [Online; accessed May 2026].
- [6] OWASP Foundation. Session management cheat sheet, 2026. [Online; accessed May 2026].
- [7] VoidZero Inc. Vitest documentation, 2026. [Online; accessed June 2026].
- [8] Meta Platforms, Inc. React documentation, 2026. [Online; accessed March 2026].
- [9] Tailwind Labs. Tailwind css documentation, 2026. [Online; accessed March 2026].
- [10] shadcn. shadcn/ui documentation, 2026. [Online; accessed March 2026].
- [11] WorkOS. Radix ui primitives documentation, 2026. [Online; accessed March 2026].
- [12] React Hook Form. React hook form documentation, 2026. [Online; accessed April 2026].
- [13] Framer. Motion for react documentation, 2026. [Online; accessed April 2026].
- [14] npnm. npnm documentation, 2026. [Online; accessed March 2026].
- [15] Microsoft Corporation. Typescript documentation, 2026. [Online; accessed March 2026].
- [16] Upstash Inc. Upstash redis documentation, 2026. [Online; accessed April 2026].
- [17] Vercel Inc. Vercel documentation, 2026. [Online; accessed April 2026].

Appendix A

Detailed Entity-Relationship Diagram

This appendix presents the full version of the Entity-Relationship Diagram introduced in Section 3.3 (Figure 3.2), including every table's columns, types, and key annotations, generated from the same Drizzle schema definitions. Relationship lines carry the same cardinality convention used in the main body: 1/N at each end, solid for `ON DELETE CASCADE`, dashed for `ON DELETE NO ACTION`.

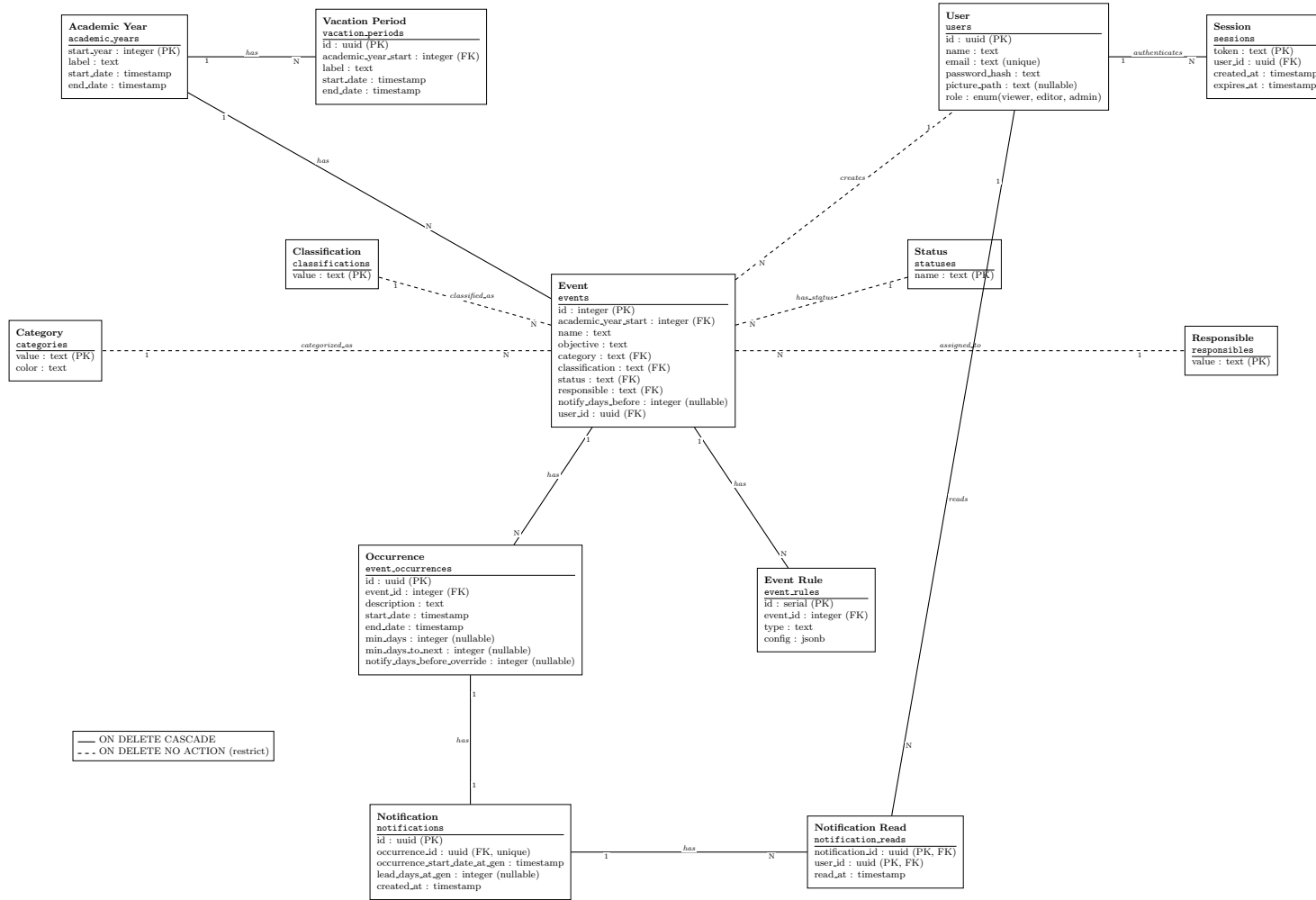


Figure A.1: Full Entity-Relationship Diagram with column-level attributes, derived from the Drizzle schema.

Appendix B

Selected Code Listings

This appendix gives the full, unabridged source of a few backend and frontend units that Chapters 4 and 5 show only in an abbreviated form, to keep those chapters within the report's page budget.

B.1 The Weekend Rule Definition

Referenced from Section 4.7.

```
1 export const weekendRule: IRuleDefinition = {
2   id: "weekend",
3   fields: [{ id: "checkStart", type: "checkbox", defaultValue: true }, { id: "checkEnd", type
4     : "checkbox", defaultValue: true }, { id: "checkRange", type: "checkbox", defaultValue:
5     false }],
6   translations: {
7     pt: {
8       label: "Fim de semana",
9       fields: { checkStart: "Verificar início", checkEnd: "Verificar fim", checkRange: "
10         Verificar período" },
11       fieldLabels: { start: "Início", end: "Fim", range: "Período" },
12       messages: {
13         startIsWeekend: "O início coincide com um fim de semana",
14         endIsWeekend: "O fim coincide com um fim de semana",
15         rangeHasWeekend: "O período inclui um dia de fim de semana",
16       },
17     },
18   },
19   validate(_event: IEvent, occurrence: IOccurrence, config: Record<string, unknown>):
20     IRuleViolation[] {
21     return runDayRangeCheck<true>({
22       occurrence,
23       config: config as WeekendConfig,
24       matchDay: (day) => (isWeekend(day) ? true : undefined),
25       toViolation: (field, date) => ({ fieldLabelKey: field, date, messageKey: MESSAGE_KEYS[
26         field] })),
27   });
28 }
```

Listing B.1: The Weekend rule definition, in full (server/calendar/rules/definitions/weekend.rule.ts).

B.2 The Global Rule-Validation Dispatcher

Referenced from Section 4.7.

```
1 private collectEventRuleIssues(  
2   event: IEvent,  
3   academicYearStart: number,  
4   allEvents: IEvent[],  
5   vacations: IVacationPeriod[] = [],  
6 ): ICalendarRuleIssue[] {  
7   const context = { allEvents, academicYearStart, vacations };  
8   const issues: ICalendarRuleIssue[] = [];  
9  
10  const sortedOccurrences = [...event.occurrences].sort(  
11    (a, b) => new Date(a.startDate).getTime() - new Date(b.startDate).getTime(),  
12  );  
13  
14  const pushIssues = (occurrence: (typeof sortedOccurrences)[number], ruleType: string,  
15    violations: IRuleViolation[]) => {  
16    for (const v of violations) {  
17      issues.push({  
18        eventId: event.id,  
19        eventName: event.name,  
20        occurrenceId: occurrence.id,  
21        occurrenceDescription: occurrence.description,  
22        academicYearStart,  
23        date: v.date,  
24        ruleType,  
25        fieldLabelKey: v.fieldLabelKey,  
26        messageKey: v.messageKey,  
27        messageParams: v.messageParams,  
28      });  
29    }  
30  };  
31  
32  for (const occurrence of sortedOccurrences) {  
33    for (const builtInRule of [minDurationRule, minDaysToNextRule]) {  
34      pushIssues(occurrence, builtInRule.id, builtInRule.validate(event, occurrence, {},  
35        context));  
36    }  
37  
38    for (const rule of (event.rules ?? [])) {  
39      pushIssues(occurrence, rule.type, validateEventRule(rule, event, occurrence, context));  
40    }  
41  }  
42  return issues;  
43 }
```

Listing B.2: The global rule-validation dispatcher, in full (server/calendar/services/calendar.service.ts).

B.3 Password Hashing and Verification

Referenced from Section 4.2.

```
1 export const hashPassword = (password: string): string => {
2   const salt = crypto.randomBytes(16).toString("hex");
3   const hash = crypto.scryptSync(password, salt, 64).toString("hex");
4   return `scrypt${salt}${hash}`;
5 };
6
7 export const verifyPassword = (password: string, encodedHash: string): boolean => {
8   const [algorithm, salt, storedHash] = encodedHash.split("$");
9   if (algorithm !== "scrypt" || !salt || !storedHash) {
10    return false;
11  }
12
13  const computedHash = crypto.scryptSync(password, salt, 64).toString("hex");
14  if (computedHash.length !== storedHash.length) return false;
15  return crypto.timingSafeEqual(
16    Buffer.from(storedHash, "hex"),
17    Buffer.from(computedHash, "hex"),
18  );
19 };
```

Listing B.3: Password hashing and verification, in full (server/auth/crypto.ts).

B.4 The withRolePolicy Authorization Wrapper

Referenced from Section 4.4.

```
1 export function withRolePolicy<
2   TInstance extends object,
3   TPolicy extends Partial<Record<keyof TInstance, TRolePolicy>>,
4 > (
5   instance: TInstance,
6   policy: TPolicy,
7   getRole: (args: unknown[]) => TUserRole,
8 ): { [K in keyof TPolicy]: TInstance[K & keyof TInstance] } {
9   const wrapped: Record<string, unknown> = {};
10
11   for (const key of Object.keys(policy)) {
12     const minimum = policy[key as keyof TPolicy] as TRolePolicy;
13     const original = instance[key as keyof TInstance] as unknown as
14       (...args: unknown[]) => unknown;
15
16     wrapped[key] = (...args: unknown[]) => {
17       if (minimum !== "any") {
18         assertRole(getRole(args), minimum);
19       }
20       return original.apply(instance, args);
21     };
22   }
23
24   return wrapped as { [K in keyof TPolicy]: TInstance[K & keyof TInstance] };
25 }
```

Listing B.4: The withRolePolicy authorization wrapper, in full (server/shared/authorize.ts).

B.5 The DraggableEvent Component

Referenced from Section 5.3.4.

```
1 export function DraggableEvent({
2   event,
3   occurrence,
4   children,
5   className,
6 }: DraggableEventProps) {
7   const { canEditEvents } = useCalendar();
8   const { startDrag, endDrag, isDragging, draggedOccurrence } = useDragDrop();
9
10  if (!canEditEvents) {
11    return <>{children}</>;
12  }
13
14  const isCurrentlyDragged = isDragging && draggedOccurrence?.occurrence.id === occurrence.id
15    ;
16
17  const handleClick = (e: React.MouseEvent<HTMLDivElement>) => {
18    e.stopPropagation();
19  };
20
21  return (
22    <motion.div
23      className={` ${className} || ""} ${isCurrentlyDragged ? "opacity-50 cursor-grabbing" : "
24        cursor-grab"} `}
25      draggable
26      onClick={(e: React.MouseEvent<HTMLDivElement>) => handleClick(e)}
27      onDragStart={(e) => {
28        (e as DragEvent).dataTransfer!.setData(
29          "text/plain",
30          occurrence.id,
31        );
32        startDrag(event, occurrence);
33      }}
34      onDragEnd={() => {
35        endDrag();
36      }}
37    >
38      {children}
39    </motion.div>
40  );
41 }
```

Listing B.5: The DraggableEvent wrapper component, in full (features/calendar/dnd/draggable-event.tsx).

Appendix C

Additional Screenshots

This appendix gives full-size screenshots referenced from Chapter 5 but not shown inline there, to keep that chapter within the report’s page budget.

C.1 Calendar Views

Referenced from Section 5.3.4. The Month view is shown inline as Figure 5.2; the remaining four are given here in full size.

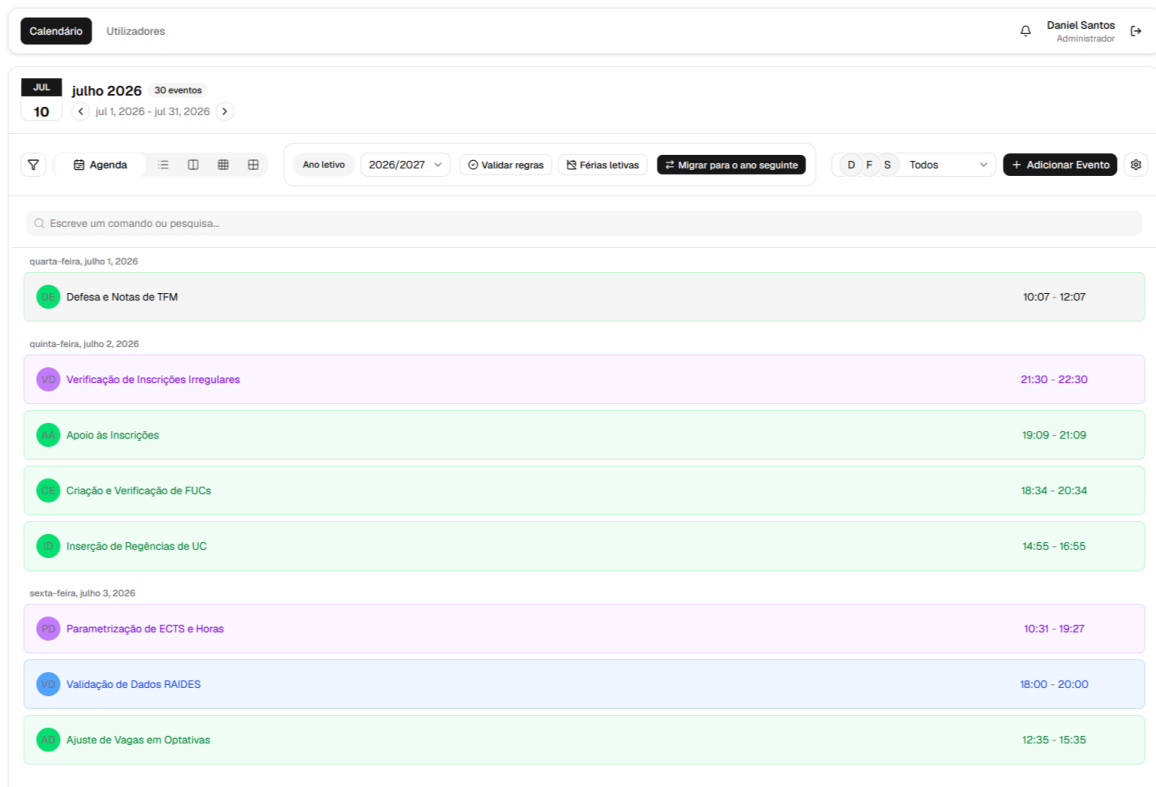


Figure C.1: Calendar interface, Agenda view: a scrollable list of occurrences grouped by date or category.

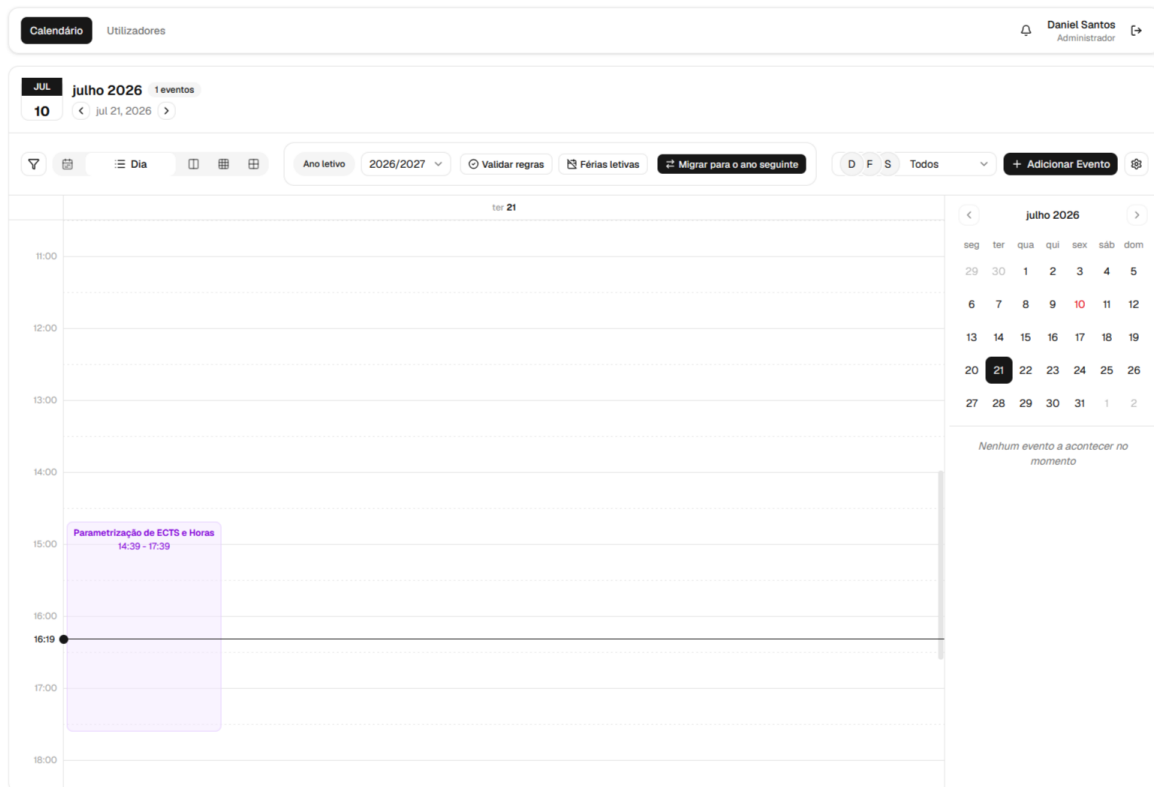


Figure C.2: Calendar interface, Day view: a single-day time grid supporting drag-and-drop rescheduling and resizing.

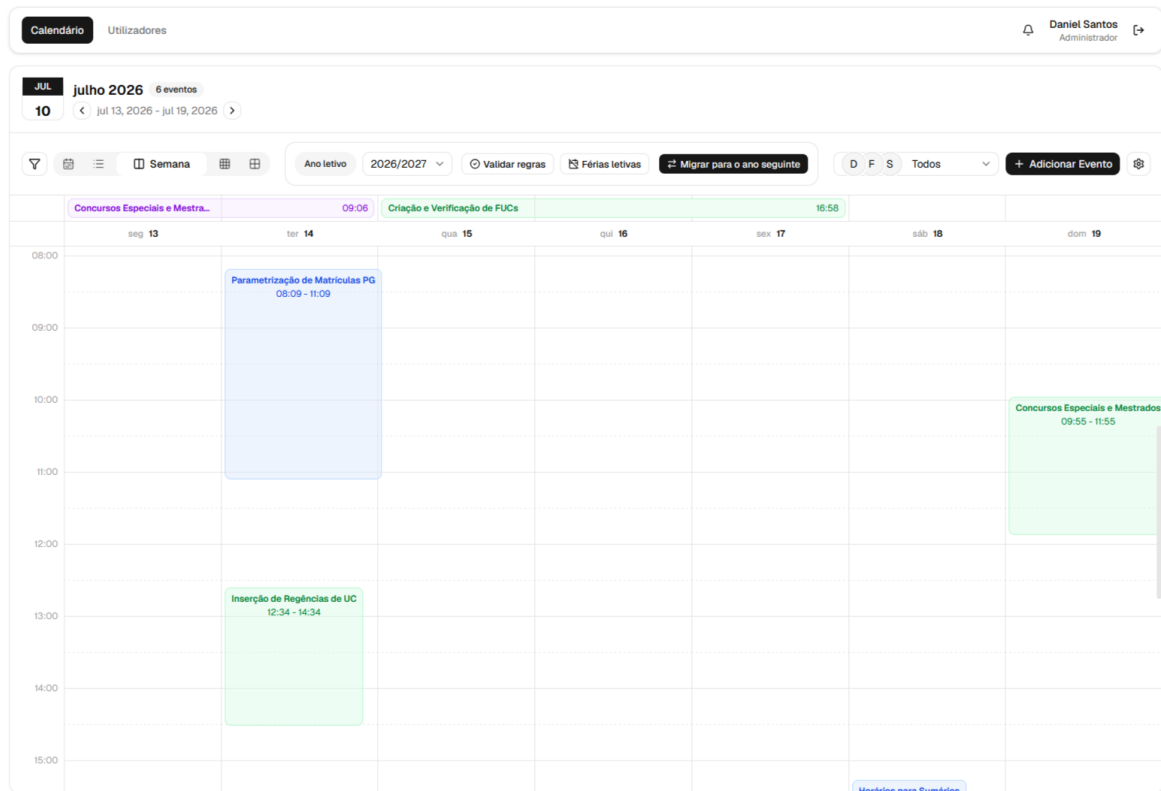


Figure C.3: Calendar interface, Week view: the same time-grid interactions as the Day view, spanning a full week.

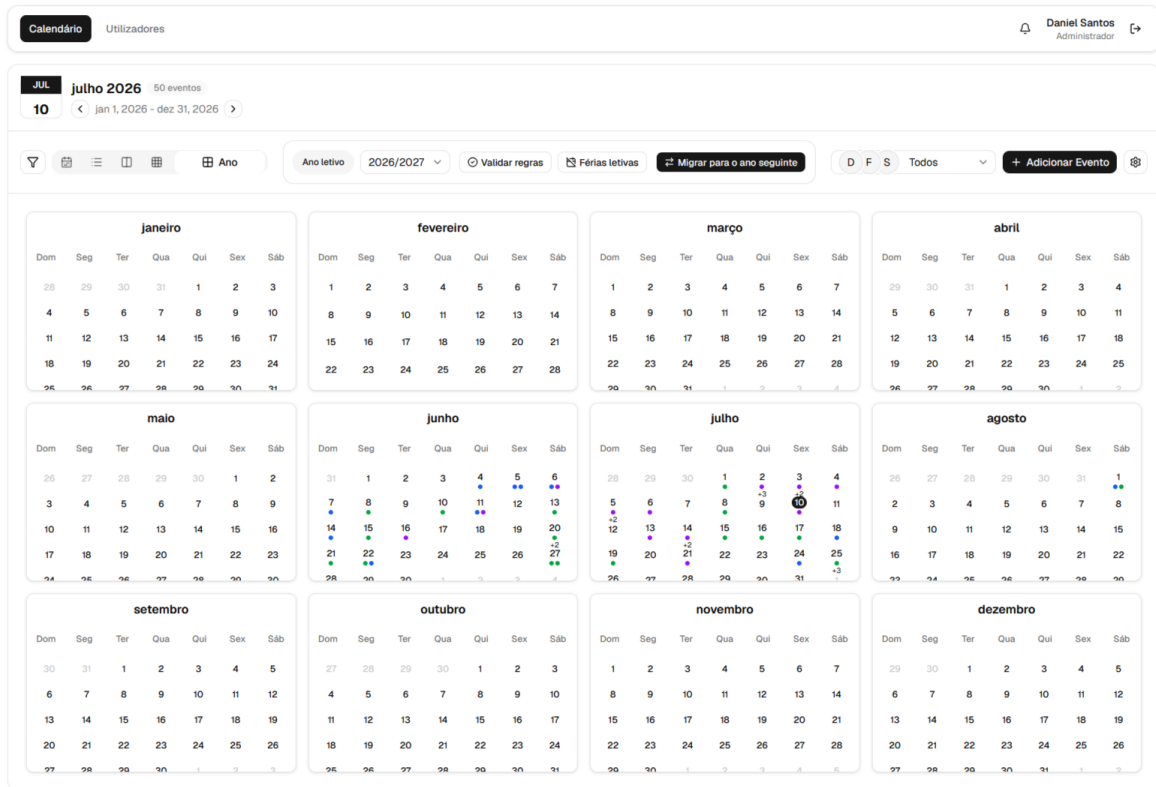


Figure C.4: Calendar interface, Year view: a full-year grid overview of the academic calendar.

C.2 Validation/Migration Results Modal

Referenced from Section 5.3.6.

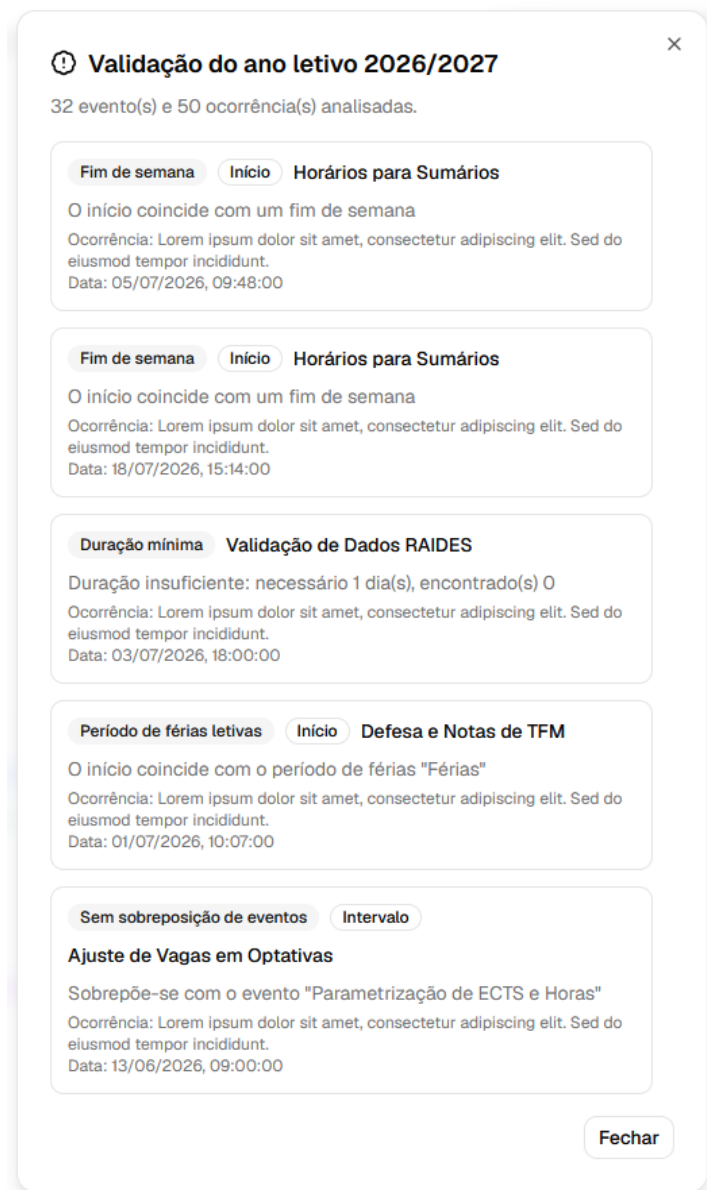


Figure C.5: Validation/migration results modal: a confirmation banner when no rule violations are found, or one card per violation, each naming the rule, the affected event/occurrence, and the offending date.